基于WebSocket的实时协同编辑方法及应用

陈诗雨,熊才权,董 奕,梅 清

(湖北工业大学 计算机学院,湖北 武汉 430068)

摘 要:操作意图一致性控制是协同编辑的重要研究问题。传统的协同控制算法在操作意图一致性和交互性上存在加锁粒度大小难以把握、函数转换结果与用户意图不一致、操作对象全局ID难以分配等局限性。为减少用户协同工作中出现的操作意图不一致现象,提高工作效率,提出一种基于WebSocket协议及隐藏/恢复机制的共享同步控制方法。该方法采用WebSocket技术,通过集合类对编辑样本和操作主体进行共同控制,从而减小操作主体的等待时间,实现在静默和动态两种情况下共享编辑样本的一致性,不仅很好解决了用户操作意图一致性问题,还能满足用户在协同过程中的交互需求,提高页面质量。

关键词:WebSocket;操作意图一致性;页面质量;共享同步控制方法(SSCM)

DOI:10. 11907/rjdk. 182139

中图分类号:TP301 **文献标识码**:A **文章编号**:1672-7800(2019)005-0079-05

Real-Time Collaborative Editing Method Based on WebSocket and Its Application

CHEN Shi-Yu, XIONG Cai-Quan, DONG Yi, MEI Qing

(*School of Computer Science*, *Hubei University of Technology*, *Wuhan* 430068, *China*)

Abstract: Operation intention consistency control is an important research problem in collaborative editing. Traditional collaborative control algorithm has shortcomings in operating intention consistency and interaction, such as the difficulties to ensure the sizes of lock granularities, the inconsistency between function conversion results and users' intentions and difficulties to allocate global operation ID. In order to reduce the user operation intent inconsistent phenomenon occurring in a collaborative work and improve the efficiency of users to work together, we put forward shared SSCM (Shared Synchronization Control Method) based on the WebSocket protocol and hide/recovery mechanism, the method uses the WebSocket technology, control the common edit samples and operating subject through the collection class, thus reduces the waiting time for operation main body, and realises the consistency of shared editing samples in silence and dynamic cases. This method can not only solve the problem of the consistency of user operation intention, but also can meet the user´s interactive demand in the collaborative process and improve the quality of the page.

Key Words: WebSocket;operational intent consistency;page quality;shared synchronization control method (SSCM)

0 引言

协同编辑指在网络环境下多人共同编辑一个文档,集结群体智慧,使最终文档满足群体多数人员的意见要求,是计算机协同工作研究领域的典型应用。协同工作要求很强的实时性,即用户不用刷新页面就能及时看到他人所作的修改。实时协同编辑需要解决的重点问题是保证用户操作意图的一致性,即如何在较高的网络延迟下尽可能多地维护用户意图。现有的操作意图一致性算法如OT(Operational Transformation)算法[1]、AST(Address SpaceTransformation)算法[2]、CRDT(Commutative Replicated Data Type)算法[3-6]等各具优点,但都存在局限性[3]。在OT算法中,本地操作可以立即执行,而远程操作需要结合本地历

收稿日期:2018-07-23

基金项目:国家重点研发计划项目(2017YFC1405403);国家自然科学基金项目(61075059);湖北工业大学绿色工业科技引领计划(产品研发类)项目(CPYF2017008);湖北省大学生创新创业训练计划项目(201710500070)

作者简介:陈诗雨(1997-),女,湖北工业大学计算机学院学生,研究方向为人工智能、软件工程;熊才权(1966-),男,博士,湖北工业大学计算机学院教授,研究方向为模式识别与智能系统、非单调推理、辩论模型;董奕(1996-),男,湖北工业大学计算机学院学生,研究方向为人工智能、软件工程;梅清(1979-),男,湖北工业大学计算机学院讲师,研究方向为模式识别与智能系统。本文通讯作者:熊才权。

史操作,进行并发操作转换后才能执行。而设计一个有效的并发操作转换函数面临很大挑战,因而这种算法在实际应用中存在一定的局限性[3]。AST(Address SpaceTransformation)算法通过文档的地址空间回溯到操作产生时的状态,以此获取操作的执行位置。为保证回溯的正确性,必须保存上次运算之后的求解路径,而数据量过大数据库就会产生超负现象。CRDT算法虽不需要保存操作历史,且并发操作之间不需要进行转换,但当操作主体很多时,操作主体标识可能不唯一,从而出现编辑冲突[3]。编辑锁是解决编辑冲突最简单的方法,但传统的协同编辑方法是对整篇文章加锁,粒度过大,不能实现多人协作;毛启容等[7]提出的乐观锁技术虽减小了加锁粒度,但却需要在加锁时人为发起请求,请求同意后才允许编辑,效率很低。无论是对整篇文章加锁还是乐观锁技术,都是基于锁或序列化的处理机制,着重于实现结果一致性却无法满足用户在协同过程中对操作意图的一致性需求。何发智等[5]对CRDT算法进行了大量优化和改进,提出了一种支持对字符串操作的集成和选择性撤销算法,以缓解选择性撤销时出现的编辑冲突。LIU等[8]提出的DSGOT算法,虽然通过逻辑时钟解决了操作之间的矛盾,可有效保证操作意图一致性,但尚未在实际文本协同编辑系统中得到应用。Sun等[9]提出的CCR方法,不仅提供了一种解决实时协同编辑系统冲突的方法,而且提供了一个支持部分冲突解决策略的框架。

随着互联网技术的快速发展,基于Web的协同编辑应用需求也越来越多。页面质量被认为是协同编辑的结果,不同层次的冲突对页面质量的影响不同,且随时间变化而变化[10]。Xiao等[11]提出的基于P2P的ORGA协同编辑方法可不依赖于操作用户的终端数量,较好实现操作意图的一致性,但P2P服务需要重复建立通道,导致网络负载压力较大,对页面质量也有所影响。目前基于Web的协同编辑技术大多采用WebSocket协议,如基于Web的实时协同图形编辑系统CoWebDraw[12],通过WebSocket协议,实现了图形的实时转换且页面质量较高。

为满足操作意图一致性,提高页面质量,解决协同编辑在网络环境下编辑冲突问题,本文提出一种基于WebSocket协议及隐藏/恢复机制[13]的共享同步控制方法(SSCM),用于实现编辑样本的编辑冲突问题,以实现实时转换目的。WebSocket是一种双向通信技术,将该技术应用到协同编辑的同步控制中有以下优势:①可避免消息的被动更新,提供容错功能[14];②可改善因操作转换算法的不唯一和难以控制带来的协同工作环境下的不稳定问题,提高用户体验感[15];③可有效实现操作意图一致性,缓解编辑冲突问题,提高协同编辑效率。

1 共享同步控制方法

共享同步控制方法(Shared Synchronization Control Method,SSCM)主要用来解决操作主体间因合作和资源共享所带来的冲突问题。该方法首先通过文章预处理建立文章数据库,然后利用集合类建立在线用户队列和段落用户队列,以保证所有用户的实时性,最后在Web环境下建立基于WebSocket服务端和客户端之间的通道,实现编辑样本的共享同步控制。在WebSocket过程控制环境下,SSCM包括对文章的控制和对用户的控制。对文章的控制包括文章的预处理以及文章的分段同步,对用户的控制包括用户编辑任务排队及多段落互斥访问控制。SSCM可有效缩小由于队列等待引起的时间差,大大提升协同编辑效率。

1.1 文章预处理

文章预处理的主要工作是对文章进行分段,建立段落库。为解决操作意图的一致性,编辑样本以段为节点,将操作记录与节点进行绑定,同时利用隐藏/恢复机制实现编辑样本的一致性。首先根据上传文件及相关文件操作获取文章信息,再根据相关算法建立整体段落库,同时标识每个段落的ID。方法描述如下:

```
输入 文章名(扩展名为doc)
输出 段落集paraList
Begin
  新建编辑样本输入流命名in,构建HWPFDocument类对象doc;
      定义length,表示文章字数;
      定义paraList存储自然段的链表;
      定义paracont 自然段内容;
      FOR i 0 to length-1 by 1 do  //当定义变量i小于文章总字数时进入该循环
            Range(i,i+1,doc);//定义读取范围
CharacterRun cr = range.getCharacterRun(0); //定义扫描字符变量cr,range表示扫描范围
            char c; //用于获取下一个读取的字符;
            IF c为换行符
                  paracont+" <br>";
                  将paracont插入paraList以更新自然段链表paraList;
                  清空paracont ;
            ENDIF
      END
      返回整合后的paraList;
END
```

预处理的基本思想:在含有n个编辑段落的编辑样本中,通过现有HWPFDocument类(处理doc文档)、XWPFDocument类(处理docx文档)对编辑样本进行扫描分段,并将段落内容插入链表内,完成每个段落的ID标识。

1.2 基于TCP的Websocket共享同步控制

基于Websocket共享同步控制操作的主要对象由客户端和服务端构成,图1为协同控制原理。

1.2.1 共享同步

Map集合类用于存储键值对,其中每个键唯一映射一个值。实现共享同步主要依赖两个不同的Map数据结构,分别命名为PassageMap和OnEditMap以示区别,结合Map集合类和WebSocket技术以实现操作意图的一致性。

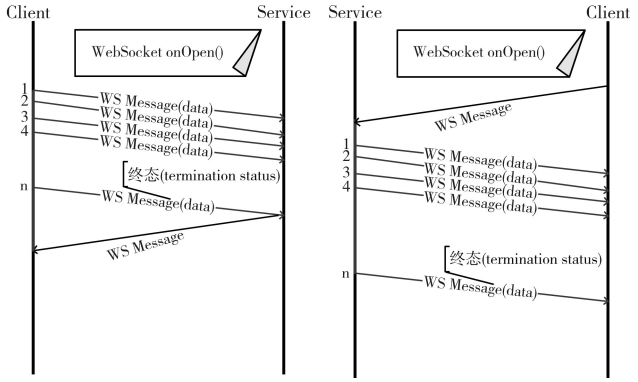

图 1　协同工作控制原理

定义 1　在线编辑用户队列。PassageMap 将所有编辑样本与样本段落队列联系起来，实现从编辑样本 ID 到段落队列的映射。每个段落队列对应一个在线编辑用户队列，以表示每个段落对应的所有编辑用户，其结构为：

$$PassageMap::\{Map<Integer, Map<Integer, Queue>>\}$$

定义 2　正在编辑用户表。OnEditMap 实现从编辑样本 ID 到正在编辑文章的映射。每个在线编辑用户队列的第一用户进入正在编辑用户表，以表示当前正在对样本进行所有用户的编辑，其结构为：

$$OnEditMap::\{Integer, List\}$$

如果各进程互斥地访问同一资源，这一资源便称为临界资源。将编辑样本中的每一段都看作一个临界资源，则协同工作过程可视为各用户排队对临界资源进行编辑，同时前一用户的编辑结果会同步更新至所有在线编辑用户界面上。

为理解同步控制过程的处理机制，这里定义 3 种状态：

定义 3　就绪状态。将处于 PassageMap 队列中的用户从第二个起至最后一个用户的状态称为就绪状态。

定义 4　执行状态。将处于 OnEditMap 中正在编辑的用户状态称为执行状态。

每个就绪队列的第一个用户加入 OnEditMap，且每次都由该用户对映射段落进行加锁，其余选择编辑该段的用户进入就绪队列处于就绪状态。处于 OnEditMap 的用户编辑完成或者超过逻辑时钟后便退出 OnEditMap，由后一处于就绪状态的用户加入 OnEditMap，状态由就绪状态变为执行状态，过程中若有用户申请编辑该段将排至就绪队列队尾，直到就绪队列用户全部退出，该段编辑结束。

定义 5　等待状态。为解决用户持续占用导致其余用户饥饿的情况，为每个 OnEditMap 中的用户添加一个 10min 的逻辑时钟（Clock），当逻辑时钟报警后该用户被提醒并存入另一队列，该队列称为等待队列，处于该队列中的用户状态称为等待状态。当处于就绪队列的第二个用户未能在第一个用户完成编辑后就进入 OnEditMap，亦被加入等待队列进入等待状态。

几种队列状态转换如图 2 所示。

设文章 A 有 n 个段落，记为 $A=\{P_1, P_2, \cdots, P_n\}$。设有 m 个用户参与协同编辑，记为 $U=\{U_1, U_2, \cdots, U_m\}$。为每个段落定义一个队列（称为用户就绪状态队列）用于用户排队，n 个段落共有 n 个队列，记为 $UQ=\{UQ_1, UQ_2, \cdots, UQ_n\}$。用 $E(UQ_i, P_i)$ 表示队列 UQ_i 中的用户对段落 P_i 进行编辑处理，（$i\in\{1, 2, \cdots, n\}$）：①若 U_j 正在编辑 P_i，则 $U_j\xrightarrow{e}P_i$，$U_j\xrightarrow{in}UQ_i$，$j\in\{1, 2, \cdots, m\}$，$i\in\{1, 2, \cdots, n\}$；②若 U_j 正在等待编辑 P_i，则 $U_j\xrightarrow{w}P_i$，$U_j\xrightarrow{in}UQ_i$；③若 U_j 完成对 P_i 的编辑，则 $U_j\xrightarrow{f}P_i$，$U_j\xrightarrow{out}UQ_i$；④若 U_j 在该篇文章的任意段落都处于等待状态，则 $U_j\xrightarrow{\tau}P$；⑤若 $\{U_1, U_2\}$ 在时间 T_1 段编辑同一段 P_1，则 $(U_1, U_2)\rightarrow(T_1, UQ_1)$。

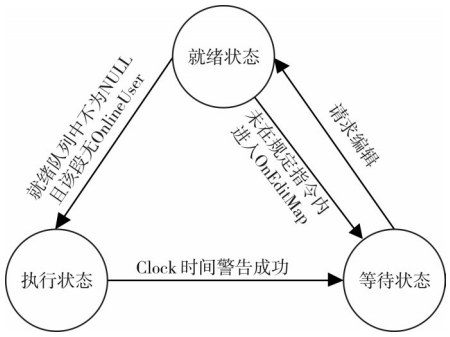

图 2　用户编辑状态转换

编辑后的文章记为 A'，则有：

$$A'=\sum_{i=1}^{n}E(UQ_i, P_i)$$

假设一篇文章有 3 个段落 $A=\{P_1, P_2, P_3\}$，对应的 PassageMap 中队列为 $UQ=\{UQ_1, UQ_2, UQ_3\}$，有 4 个用户 $U=\{U_1, U_2, U_3, U_4\}$ 对该文章进行编辑，如图 3 所示。每个队列的第一个元素表示正在编辑该段落的用户。

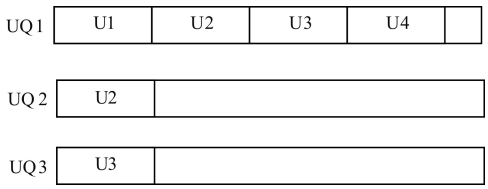

图 3　用户就绪队列

U_1 是 UQ_1 的第一个元素，则 $U_1\xrightarrow{e}P_1$，接着 U_2 进入 UQ_1，则 $U_2\xrightarrow{w}P_1$。但等待过程中 U_2 仍可选择编辑其它段落，例如此时 U_2 进入 UQ_2，为 UQ_2 的第一个元素，则可直接编辑 $U_2\xrightarrow{e}P_2$。同理，若 U_3 也要编辑第一段，则 $U_3\xrightarrow{w}P_1$，若在等待过程中进入 UQ_3，则 $U_3\xrightarrow{e}P_3$。

此外，用 OnEditMap 存储正在编辑段落的用户，如图 4 所示，即满足 $U_j\xrightarrow{e}P_i$，$U_j\xrightarrow{in}UQ_i$；$j\in\{1, 2, \cdots, m\}$，$i\in\{1, 2, \cdots, n\}$ 的用户都被存储在该链表中。

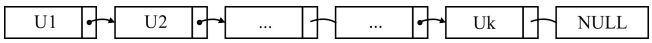

图 4　OnEditMap 结构（0<k<n）

当 $U_1 \xrightarrow{f} P_1$，则 U_2 的状态由 $U_2 \xrightarrow{w} P_1$ 更改为 $U_2 \xrightarrow{e} P_1$，但若此时 $U_2 \xrightarrow{e} P_2$，则扫描该链表寻找第一个 $U_j \xrightarrow{\tau} P_i$ $(j \in \{1, 2, \cdots, m\}, i \in \{1, 2, \cdots, n\})$，$Swap(U_i, U_2)$，如图5所示。

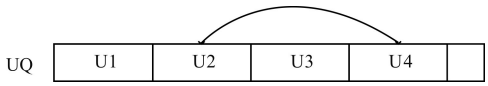

图5 $Swap(U_i, U_2)$

当队列中所有用户都在编辑其它段落时，需等待排在最前面用户一定时间后，将第一个用户排至队尾再等待下一个用户，例如当 $(U_1 \xrightarrow{f} P_1) \wedge (U_2 \xrightarrow{w} P_1) \wedge (U_2 \xrightarrow{w} P_1) \wedge \cdots$，即未找到 $U_i \in (U_j \xrightarrow{\tau} P_i), j \in \{1, 2, \cdots, m\}, i \in \{1, 2, \cdots, n\}$，那么等待 U_2 一定时间后将 U_2 排至队尾，再等待 U_3 一定时间，依次类推。对应用户都可在等待时间内结束目前编辑任务，例如正在编辑其它段落的 U_2 可在等待时间内结束正在编辑的段落。

用户结束编辑后会从相应段落的队列中弹出，同时也会从正在编辑段落的用户链表中移除，并将队列中的下一位用户插入正在编辑段落的用户链表，从而在满足协同工作一致性的基础上提高协同效率。

1.2.2 操作意图一致性控制

操作转换的基本思想是本地操作立即执行，远程操作经过转换之后再执行，各个站点按照不同的顺序执行操作后获得相同的结果[16]。但操作转换算法的复杂度会直接影响到协同工作效率，本文对基于隐藏/恢复机制的操作转换方法[13]进行改进，同时引入WebSocket技术，以避免因算法复杂度高而造成的协同效率低下问题。客户端用于生成消息、控制测试和显示结果，服务端则进行处理和广播。为此，创建客户端界面用于承载编写的数据，Websocket用于传递客户端界面与服务器之间的数据[17]，具体处理过程如下：

服务端：对节点进行操作，将修改集成到HB（History Block）中并进行广播。

具体过程如下：①初始化段落标记，$Flag[i]=0$，$i \in \{0, 1, \cdots, N\}$；②若标记未置1，则标记更改，$Flag[i]=1$，$i \in \{0, 1, \cdots, N\}$，段落进入可编辑状态（此时其他用户处于就绪态）；③通过WebSocket技术对在线用户进行广播；④编辑结束，标记位置0，$Flag[i]=0$，$i \in \{0, 1, \cdots, N\}$，同时保存HB；⑤通过WebSocket技术对在线用户进行广播；⑥下一用户进入执行状态，重复步骤②—步骤⑥。

客户端：对节点进行操作，及时获取WebSocket数据，过程如下：①初始化状态：段落进入可编辑状态；②对段落文字进行修改；③向WebSocket发送HB消息，接收WebSocket的HB消息。

以下用图形表示各站点和各节点的操作，图6为同一篇共享文章中3个站点的交互过程，以站点2为例：

站点1发出操作O1/P1，即 $U_1 \xrightarrow{e} P_1$，WebSocket广播给各站点，各站点几乎在同一时间段内接收到消息，此阶段 P_1 为临界资源，站点2对 P_2 进行操作，即 $U_2 \xrightarrow{e} P_2$。同理，通过已建立好的WebSocket通道使其它站点接收到此操作。与此同时，站点2可接收由站点1发来的对 P_1 的操作消息，另外站点3发出 $U_3 \xrightarrow{e} P_1$ 操作不会对 $U_2 \xrightarrow{e} P_2$ 有任何影响。当 $(U_3 \xrightarrow{f} P_1) \wedge (U_2 \xrightarrow{f} P_2)$，站点2上共享文章的显示处理结果为{O1，O3，02}。同理，站点1和站点3的运行历史记录HB={O1，O3，02}。

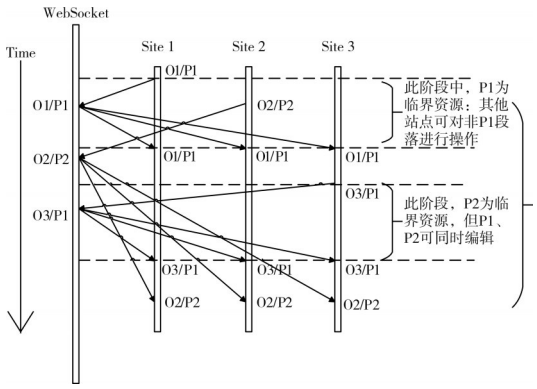

图6 基于Websocket的共享同步控制

2 案例分析

为验证本文提出的共享同步控制方法（SSCM）的可行性和高效性，以SSCM为基础设计了一个原型系统，对本文提出的方法进行测试。团队共7组学生参与协同工作，这7组学生可根据自己的偏好对编辑样本进行修改，并通过协商达成一致。实验用的编辑样本为《团队简介》，编辑前样本界面如图7所示。

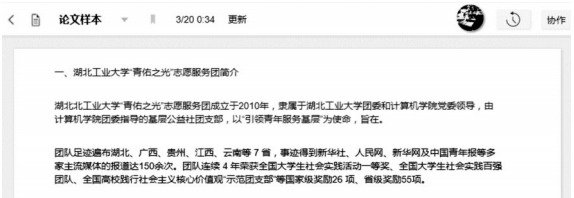

图7 原始编辑样本

各站点操作为：

站点1：{添加，（"事迹得到新华社、人民网、新华网及中国青年报等多家主流媒体的报道"）}；

站点2：{删除，（"新华网"）}；

站点3：{修改，（"得到"，"获得"）}；

站点2：{添加，（"深入引导青年学生践行社会主义核心价值观、增强青年学子对'中国梦'的认知、理解和实践。"）}；

站点4：{修改，（"中国青年报"，"中青网"）}

各站点操作顺序为：在 T_1 时间段内，$U_1 \xrightarrow{e} P_3$、$(U_1) \rightarrow (T_1, UQ_3)$ $\{U_1 \rightarrow (T_1, UQ_3)\}$；在 T_2 时间段内，(U_3, U_2, U_4)

$\rightarrow (T_2, UQ_3)$ 的同时 $(U_2) \rightarrow (T_2, UQ_2)$。

即在 T_2 时间段内，U_2、U_3、U_4 三个操作主体想要编辑第三段 P_3，$(U_3, U_2, U4) \xrightarrow{in} UQ_3$。与此同时，操作主体 U_2 想要编辑第二段 P_2，$U_2 \xrightarrow{in} UQ_2$。若按传统的加锁控制方法，则一个操作主体在一个时间段内只能排在一个队列里，即 U_2 无法在 $(U_1 \xrightarrow{e} P_3) \wedge (U_2 \xrightarrow{w} P_3)$ 时间段内执行操作 $U_2 \xrightarrow{e} P_2$，操作效率会大大降低。但采用 SSCM 方法后，U_2 在 $U_2 \xrightarrow{w} P_3$ 时间段内可直接执行操作 $U_2 \xrightarrow{e} P_2$，若此时 $(U_1 \xrightarrow{f} P_3) \wedge (U_2 \xrightarrow{e} P_2)$，则会对操作主体 U_2 进行提醒，若 U_2 不放弃操作 $U_2 \xrightarrow{e} P_1$，则 $U_3 \xrightarrow{e} P_3$，反之 $U_2 \xrightarrow{e} P_3$，即操作主体可在等待时间内对另一段落进行操作。利用此类处理机制，SSCM 不仅可以保存操作主体对将要编辑样本段落的操作，也可触发操作主体想要编辑的样本段落操作，从而实现协同工作的高效性。

在现有协同编辑中，若 T_1 时间内操作主体 U_1 编辑完后，U_2、U_3、U_4 同时对某段进行编辑会产生 3 个冲突点。即 U_3、U_4 无法看到 U_2 编辑的结果、U_2、U_4 无法看到 U_3 编辑的结果，U_2、U_3 无法看到 U_4 编辑的结果。在这种抢占式控制下，无疑会扰乱操作主体的思考方向，不利于在动态编辑中保持所有操作主体的操作意图一致性。SSCM 利用上述共享同步方法可很好地避免此类问题出现，以实现在静默和动态下共享编辑样本的一致性，见图 8、图 9、图 10。

图 8　操作主体 U_2，U_3，U_4 依次在同一时间段进入 P_3

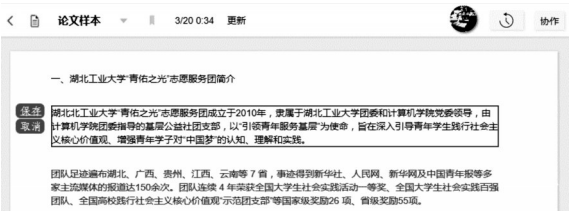

图 9　操作主体 U_2 在等待 P_3 时编辑 P_2

最终得到的一致性编辑样本见图 10。

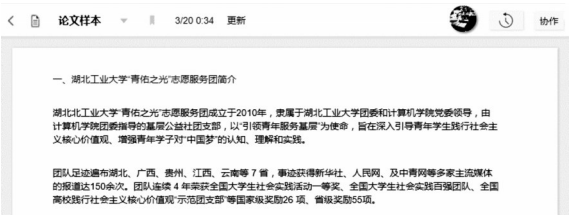

图 10　结果一致性展示

同时将多组实验数据与现有协同编辑在站点数、冲突数相同条件下进行对比，实验结果如表 1 所示。

表 1　实验数据对比

案例	站点数	冲突数	$T_{(现有协同编辑)}$	$T_{(SSCM方法)}$
1	4	3	1′19″15	1′02″01
2	4	0	2′07″02	1′55″13
3	4	1	1′10″02	0′58″22
4	4	2	0′50″39	0′35″19
5	5	1	01′18″84	01′27″69
6	7	5	02′06″95	01′58″87
7	6	6	02′24″97	02′11″34

从图 11 可以看到，当站点数相同时，冲突数越大，SCCM 方法处理的效果越好。

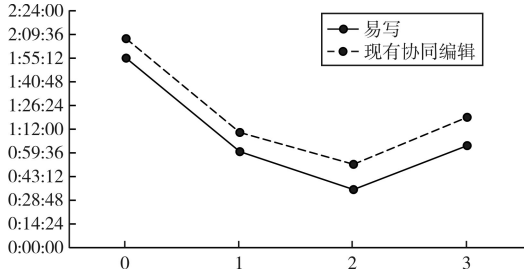

图 11　站点数相同的实验数据对比

从以上结果分析可以看出，本文提出的共享同步控制方法（SSCM）具有较高的可行性和高效性。

3　结语

本文对协同工作中并发控制方法进行了研究，提出了一种基于 WebSocket 协议及隐藏/恢复机制的共享同步控制方法（SSCM），并将协同编辑过程进行了可视化展示，能够较好地满足协同过程中的因果、结果及操作意图的一致性，同时有较强的交互性。该方法是在 WebSocket 基本原理和 3 种状态间相互转换前提下得到正在执行的线性表，可保证不同段并发操作的独立性。从另一个角度来说，客户端上的显示是通过服务端不断广播数据和 SSCM 方法中操作主体的队列转换，实现编辑样本的实时把控和主观评判，为进一步的协同作业提供参考。利用 WebSocket 的优点可以轻松解决客户端和服务端之间通道重复建立问题，使消息传递的速度提高，提升各操作主体间的交互性。另外，SSCM 中操作主体可对编辑样本进行选择操作，从而更好地集结协同工作用户的群体智慧，促进一致性达成，在采用全复制式体系结构的基础上实现一致性维护，为协同工作用户提供良好的响应性。

本文方法存在一些不足：由于方法以段落为节点，对于段间的操作存在一定的局限性。此外，由于集合本身的限制，使得部分用户无法立刻对文章段落进行操作，离协同的完全高效仍有一定距离，这是下一步研究的方向。

参考文献：

[1]　SUN C, ELLIS C. Operational transformation in real-time group edi-

（下转第 88 页）

search Society, 1988, 39(1): 41-53.

[6] BEASLEY J E. Algorithms for unconstrained two-dimensional guillotine cutting[J]. Journal of the Operational Research Society, 1985, 36(4): 297-306.

[7] SHPITALNI M, MANEVICH V. Optimal orthogonal subdivision of rectangular sheets[J]. Journal of Manufacturing Science and Engineering, 1996, 118(3): 281-288.

[8] 蒋兴波,吕肖庆,刘成城. 二维矩形条带装箱问题的底部左齐择优匹配算法[J]. 软件学报,2009,20:1528-1538.

[9] CUI Y D, YANG Y L, CHENG X, et al. A recursive branch-and-bound algorithm for the rectangular guillotine strip packing problem [J]. Computer and Operations Research, 2008, 35: 1281-1291.

[10] HUANG W Q, CHEN D B, XU R C. A new heuristic algorithm for rectangle packing[J]. Computer and Operations Research, 2007, 34: 3270-3280.

[11] BORTFELDT A. A genetic algorithm for the two-dimensional strip packing problem with rectangular pieces[J]. European Journal of Operational Research, 2006, 172: 814-837.

[12] JIANG J Q, LIANG Y C, SHI X H, et al. A hybrid algorithm based on PSO and SA and its application for two-dimensional non-guillotine cutting stock problem[J]. Lecture Notes of Computer Science, 2004, 3007: 666-669.

[13] ZHANG D F, WEI L J, LEUNG S C H, et al. A binary search heuristic algorithm based on randomized local search for the rectangular strip-packing problem [J]. Informs Journal on Computing, 2013, 25: 332-345.

[14] ALVAREZ-VALDES R, PARRENO F, TAMARIT J M. Reactive GRASP for the strip-packing problem[J]. Computers & Operations Research, 2008, 35(4): 1065-1083.

[15] SILVA E, ALVELOS F, CARVALHO J M V. Integrating two-dimensional cutting stock and lot-sizing problems[J]. Journal of the Operational Research Society, 2013, 65(1): 108-123.

[16] ANDRADE R, BIRGIN E G, MORABITO R. Two-stage two-dimensional guillotine cutting stock problems with usable leftover[J]. International Transactions in Operational Research, 2016, 23: 121-145.

[17] GOMEZ J C, TERASHIMA-MARIN H. Evolutionary hyper-heuristics for tackling bi-objective 2D bin packing problems [J]. Genet Program Evolvable Mach, 2018, 19: 151-181.

[18] 胡文蓓,饶昊. 二维 Packing 问题拟人型算法中的放置空间更新过程求解[J]. 软件导刊,2017,16(8): 19-20.

[19] ALVAREZ-VALDES R, PARREÑO F, TAMARIT J M. A GRASP algorithm for constrained two-dimensional non-guillotine cutting problems [J]. Journal of Operational Research Society, 2005, 56 (4): 414-425.

[20] ALVAREZ-VALDES R, PARREÑO F, TAMARIT J M. A TABU search algorithm for two-dimensional non-guillotine cutting problems [J]. European Journal of Operational Research, 2007, 183 (3): 1167-1182.

(责任编辑:江 艳)

(上接第 13 页)

tors: issues, algorithms, and achievements[C]. Proceedings of the 1998 ACM conference on Computer supported cooperative work. Seattle, 1998: 59-68.

[2] GU N, YANG J, ZHANG Q. Consistency maintenance based on the mark retrace technique in groupware systems[C]. 2005 International ACM SIGGROUP Conference on Supporting Group Work, Sanibel Island, 2005: 264-273.

[3] 何发智,吕晓,蔡维纬,等. 支持操作意图一致性的实时协同编辑算法综述[J]. 计算机学报,2017,41(4):1-28.

[4] MARTIN S, URSO P, WEISS S. Scalable xml collaborative editing with undo (short paper)[C]. 9th Confederated International Conferences on On the Move to Meaningful Internet Systems, 2010: 507-514.

[5] LV X, HE F Z, CAI W W, et al. Supporting selective undo of string-wise operations for collaborative editing systems [J]. Futur Gener Comp Syst. 2018 May, 82: 41-62.

[6] LV X, HE F, CAI W, et al. A string-wise crdt algorithm for smart and large-scale collaborative editing systems[J]. Advanced Engineering Informatics, 2017(33): 397-409.

[7] 毛启容,王进峰,詹永照. 相对位置乐观锁机制及在协同编辑中的应用[J]. 计算机辅助设计与图形学学报,2004,16(9):1307-1312.

[8] LIU J H, WEI G Y, WANG C. Research on concurrency control algorithm for real-time collaborative editing systems[J]. Journal of China Universities of Posts and Telecommunications, 2014, 21(SUPPL. 1): 6-11.

[9] SUN D, SUN C, XIA S, et al. Creative conflict resolution in realtime collaborative editing systems[C]. ACM 2012 Conference on Computer Supported Cooperative Work, 2012: 1411-1420.

[10] QIU J, WANG C, CUI M. The influence of cognitive conflict on the result of collaborative editing in Wikipedia[J]. Behaviour & Information Technology, 2014, 33(12): 1361-1370.

[11] LV X, LI Y, ANG C. An improved commutative replicated data type for peer to peer collaborative editing. 3rd International conference on machinery[C]. Materials and Information Technology Applications, 2015: 390-395.

[12] GAO L, GAO D, XIONG N, et al. Cowebdraw: a real-time collaborative graphical editing system supporting multi-clients based on HTML5 [J]. Multimedia Tools and Applications, 2018, 77 (4): 5067-5082.

[13] LIU J, SHAO Y, TENG G, et al. Concurrency control strategy in real-time collaborative editing system[C]. 2nd International Conference on Education Technology and Computer, 2010: V3222-V3225.

[14] QIN X. Fault-tolerant support in real-time collaborative editing systems[J]. IEEE Distributed Systems Online. 2001, 2(1): 33-45.

[15] JIANG F Y, DUAN H C. Application research of websocket technology on Web tree component[C]. 2012 International Symposium on Information Technologies in Medicine and Education, 2012: 889-892.

[16] CAI W W, HE F Z, LV X. An efficient preserving intention operational transformation for real-time collaborative editing[J]. Jisuanji Xuebao/Chinese Journal of Computers. 2015, 38(10): 2041-2053.

[17] SLODZIAK W, NOWAK Z. Based on xmlhttprequest, server-sent events and websocket[C]. 36th International Conference Information Systems Architecture and Technology, Karpacz, Poland: Springer Verlag, 2016: 71-83.

(责任编辑:杜能钢)